SECURITY & PRODUCTION AUDIT

CheapestGo
Production Readiness Report

Full-stack audit: Security · Payments · Booking Flows · Error Handling · Mobile

DATE	FRAMEWORK	BACKEND	MOBILE
March 2026	Next.js 15 App Router	Supabase + Stripe	Capacitor WebView

VERDICT

NOT PRODUCTION READY

OVERALL SCORE

4.5

out of 10

NOT Production Ready

5 blockers must be resolved before any real user traffic.

The payment foundation is solid — gaps are around access control,

data consistency under failure, and observability.

Security	Payments	Error Handling
6/10	7/10	7/10
Monitoring	**Rate Limiting**	**Data Consistency**
4/10	5/10	5/10

5	4	4	4
BLOCKERS	HIGH RISK	MEDIUM	LOW PRIORITY

PART 1 — BLOCKERS (MUST FIX BEFORE PRODUCTION)

B1 Cron Route Has Hardcoded Fallback Secret

`src/app/api/internal/refresh-flights/route.ts:15`

The Bearer token check falls back to a hardcoded string when the env var is absent:

```
if (authHeader !== `Bearer ${env.SUPABASE_SERVICE_ROLE_KEY || 'placeholder-internal-secret'}`)
```

If `SUPABASE_SERVICE_ROLE_KEY` is missing from the deployment environment, any request with `Authorization: Bearer placeholder-internal-secret` can trigger the cron job — no authentication required.

Fix: Remove the fallback entirely. If the env var is absent, return 500 immediately (misconfiguration). Introduce a dedicated `CRON_SECRET` env var and never reuse `SUPABASE_SERVICE_ROLE_KEY` as a cron token.

B2 /api/email Endpoint Has No Authentication

`src/app/api/email/route.ts`

The route accepts any POST request and immediately sends a booking confirmation email — no `getUser()` check, no session validation:

```
export async function POST(req: Request) {
    const body = await req.json();
    const data = await sendBookingConfirmationEmail(body);
    return Response.json(data);
}
```

An attacker can send thousands of emails to arbitrary addresses, constituting spam abuse and a data privacy violation. **Fix:** Add `supabase.auth.getUser()` and return 401 if unauthenticated. Additionally, verify the target email matches the authenticated user's email to prevent impersonation.

B3 Hotel Booking Race Condition — Orphaned Booking Risk

`src/components/checkout/CheckoutContent.tsx:220-235`

The hotel booking confirmation flow has two sequential steps: (1) call LiteAPI to confirm the reservation, then (2) insert the booking record into the database. If the user closes the browser, the network drops, or the server crashes between steps, the booking is confirmed with LiteAPI (real money charged, room held) but does not exist in the database.

Fix: Write a `status: 'pending'` DB record *before* calling LiteAPI. Update it to `'confirmed'` after success. This ensures any orphaned booking is visible and recoverable. Add a background reconciliation job to detect pending records older than 5 minutes and alert staff.

B4 Zero-Dollar Flight Bookings Possible

`src/app/api/flights/book/route.ts:104–106`

Price revalidation explicitly skips a `newPrice === 0` result and continues with the original price — but there is no hard floor validation on `flightTotal` itself. A manipulated flight payload with `price: 0` would cause Stripe to create a $0 PaymentIntent, completing a "free" booking.

```
// Missing floor check – add before Stripe PaymentIntent creation:
if (flightTotal ≤ 0) {
    return NextResponse.json({ error: 'Invalid price – must be > $0' }, { status: 400 });
}
```

B5 Admin Role Check is TOCTOU Vulnerable

`src/lib/server/admin/auth.ts:24–28`

Every admin API call fetches the user's role from the `profiles` table at the start of the request via `requireAdmin()`. Between that check returning and the destructive action executing, an administrator's role can be revoked in Supabase (via dashboard or direct SQL). The in-flight request has no knowledge of this and continues with full admin privileges — a classic Time-of-Check / Time-of-Use (TOCTOU) race.

Fix: Enforce role-based access at the Postgres level using Row Level Security (RLS) policies on sensitive tables. Database-level enforcement is the only backstop that is not susceptible to application-layer race conditions.

PART 2 — HIGH RISK ISSUES

H1 No Audit Log for Admin Actions

`src/app/api/admin/* (all routes)`

Destructive admin operations — ban user, force refund, cancel booking — are logged only to `console.error()`. There is no persistent audit trail. A compromised admin account could perform irreversible actions (bulk refunds, user deletions) with no forensic record to investigate.

Fix: Insert into an `admin_audit_logs` table on every mutation: `admin_id`, `action`, `target_id`, `reason`, `ip_address`, `timestamp`.

H2 Stripe Webhook Has No Event Deduplication

`src/app/api/webhooks/stripe/route.ts`

Stripe retries webhooks with exponential backoff. Two copies of the same event arriving in quick succession are both processed fully — redundant LiteAPI calls, duplicate email sends, and log noise. While the Edge Function is idempotent enough to prevent double-booking, the side effects are not.

Fix: Track processed `event.id` in a `stripe_webhook_events` table. Return 200 immediately if the event ID was already processed.

H3 Rate Limiter is In-Memory — Non-Persistent

`src/app/api/voucher/validate/route.ts:6–38`

The voucher validation rate limiter stores attempt counters in a `Map<string, number[]>` in application memory. The counter resets on every deploy or server restart. On a multi-replica Docker setup, each instance has its own independent counter — an attacker gets *N* × *limit* attempts across N instances.

Fix: Replace with a Redis-backed or Supabase-backed distributed counter that persists across restarts and is shared across all instances.

H4 No Rate Limiting on Auth / Password Reset Endpoints

No visible brute-force protection on sign-in attempts or password reset requests beyond Supabase's default configuration. An attacker can enumerate valid email addresses by spamming the password reset endpoint and observing timing differences in responses.

Fix: Configure Supabase Auth rate limits in the dashboard: max 5 password reset requests per email per hour, max 10 sign-in failures per IP per 15 minutes.

PART 3 — MEDIUM ISSUES

M1 Error Messages Leak Internal Details in Production

`Multiple API routes – e.g. src/app/api/autocomplete/route.ts`

Several API routes return `String(err)` directly in the JSON response body. In production, this exposes stack traces, internal API URLs, database structure, and third-party library versions to clients.

```
// Current (leaks stack trace in prod)
return Response.json({ success: false, error: String(err) }, { status: 500 });

// Fix
const message = process.env.NODE_ENV === 'production'
    ? 'Something went wrong. Please try again.'
    : String(err);
return Response.json({ success: false, error: message }, { status: 500 });
```

M2 No Input Length Validation on API Routes

`src/app/api/autocomplete/route.ts and others`

Routes accepting user-supplied string parameters impose no maximum length. An attacker can send a 10 MB request body, causing high CPU usage during parsing, memory pressure, and potential request timeouts. This is a cheap denial-of-service vector.

Fix: Validate input length at every route boundary. For search/autocomplete: cap query strings at 200 characters. For booking payloads: use Zod `.max()` constraints on all string fields.

M3 CSP Allows unsafe-inline and unsafe-eval

`next.config.mjs:48`

The Content Security Policy header includes both `'unsafe-inline'` and `'unsafe-eval'`. These two directives effectively disable the primary XSS defence that CSP is designed to provide. If any XSS vector exists in the application, an attacker can execute arbitrary JavaScript in the user's browser, steal session tokens, and impersonate users.

```
// Current (XSS-unsafe)
"script-src 'self' 'unsafe-eval' 'unsafe-inline' blob: https://js.stripe.com ..."

// Target: remove unsafe-inline/unsafe-eval; use nonce-based CSP for Stripe + Mapbox
```

M4 Session Expiry Not Handled During Multi-Step Checkout

`src/components/checkout/CheckoutContent.tsx`

Checkout spans multiple steps (form entry → Stripe → confirmation) and can take several minutes. If the Supabase session expires mid-flow, API calls silently return 401. The user sees a broken state with no recovery path, and may believe their payment failed even if it succeeded.

Fix: Check session validity before each step transition. On expiry, preserve booking state in `sessionStorage`, redirect to login with a `?next=/checkout` param, and restore state after re-authentication.

PART 4 — LOW PRIORITY

#	ISSUE	FILE	FIX
L1	Hardcoded fallback Vercel URL	`src/utils/env.ts:30`	Require `NEXT_PUBLIC_SITE_URL` — no fallback. Fail fast on misconfiguration.
L2	No structured logging	All API routes	Emit JSON-structured logs (`_event`, `userId`, `timestamp`) for booking_confirmed, payment_failed, booking_orphaned.
L3	Flight search results not cached	`src/app/api/flights/search/route.ts`	Cache identical searches in Redis with 30-second TTL to reduce Duffel/Mystifly API costs.
L4	Admin dashboard stale time 2 min	`src/lib/queryClient.ts`	Lower to 30s or add a manual refresh button so cancelled bookings reflect sooner.

🔐 SECURITY SUMMARY

✅ Strengths

- ✅ Supabase SSR middleware correctly protects `/admin` routes
- ✅ Stripe webhook verified with `constructEvent()`
- ✅ Service role key never exposed to client components
- ✅ Critical booking routes use Zod schema validation
- ✅ Idempotency keys present on flight bookings
- ✅ Comprehensive HTTP security headers (HSTS, X-Frame-Options, etc.)
- ✅ Admin role verified on every request via `requireAdmin()`

🚨 Critical Gaps

- 🚨 Unauthenticated email endpoint — arbitrary spam possible
- 🚨 Hardcoded cron secret fallback — unauthorized job execution
- 🚨 No RLS enforcement as database-level backstop
- 🚨 In-memory rate limiter bypassed on multi-instance deploy
- 🚨 CSP `unsafe-inline/unsafe-eval` negates XSS protection
- 🚨 No audit trail for admin destructive actions

💳 PAYMENT SAFETY VERDICT

FLOW	VERDICT	NOTES
Flights Stripe + Duffel/Mystifly	✅ SAFE	Webhook signature verified. Session/payment linkage validated. Idempotency keys prevent double-booking. **One gap:** no price floor — add `if (flightTotal <= 0) reject`.
Hotels Stripe + LiteAPI	⚠️ PARTIAL	Payment verified before LiteAPI call. Auto-refund on LiteAPI failure. **Critical gap:** if DB save fails after LiteAPI confirms, payment is charged with no DB record. Fix: write pending record before calling LiteAPI.

🚀 PRODUCTION READINESS SCORE

AREA	SCORE	KEY ISSUES
Security	**6 / 10**	Good auth foundation; unauthenticated email endpoint, no RLS backstop, weak CSP
Payment Safety	**7 / 10**	Stripe solid; hotel booking race condition is the main gap
Error Handling	**7 / 10**	Proper try/catch coverage; error messages leak internal details in production
Monitoring / Logging	**4 / 10**	console.log only; no structured logs, no audit trail, no alerting
Rate Limiting	**5 / 10**	Voucher rate limit exists but non-persistent; auth endpoints unprotected
Data Consistency	**5 / 10**	Hotel booking orphan risk; webhook deduplication missing
Overall	**4.5 / 10**	**NOT production ready. Fix 5 blockers minimum before real users.**

Minimum viable launch checklist

B1 — Remove hardcoded cron secret fallback
B2 — Add auth guard to `/api/email`
B3 — Write pending DB record before LiteAPI confirm call
B4 — Add price floor check: `if (total <= 0) return 400`
B5 — Enforce admin access via Postgres RLS policies
H3 — Replace in-memory rate limiter with Redis/Supabase-backed counter

With these 6 fixes applied, the score rises to approximately **7.5 / 10** — viable for a limited beta launch with monitoring in place.